1장. 자바 시작하기

이것이 자바다



Contents

❖ 목차

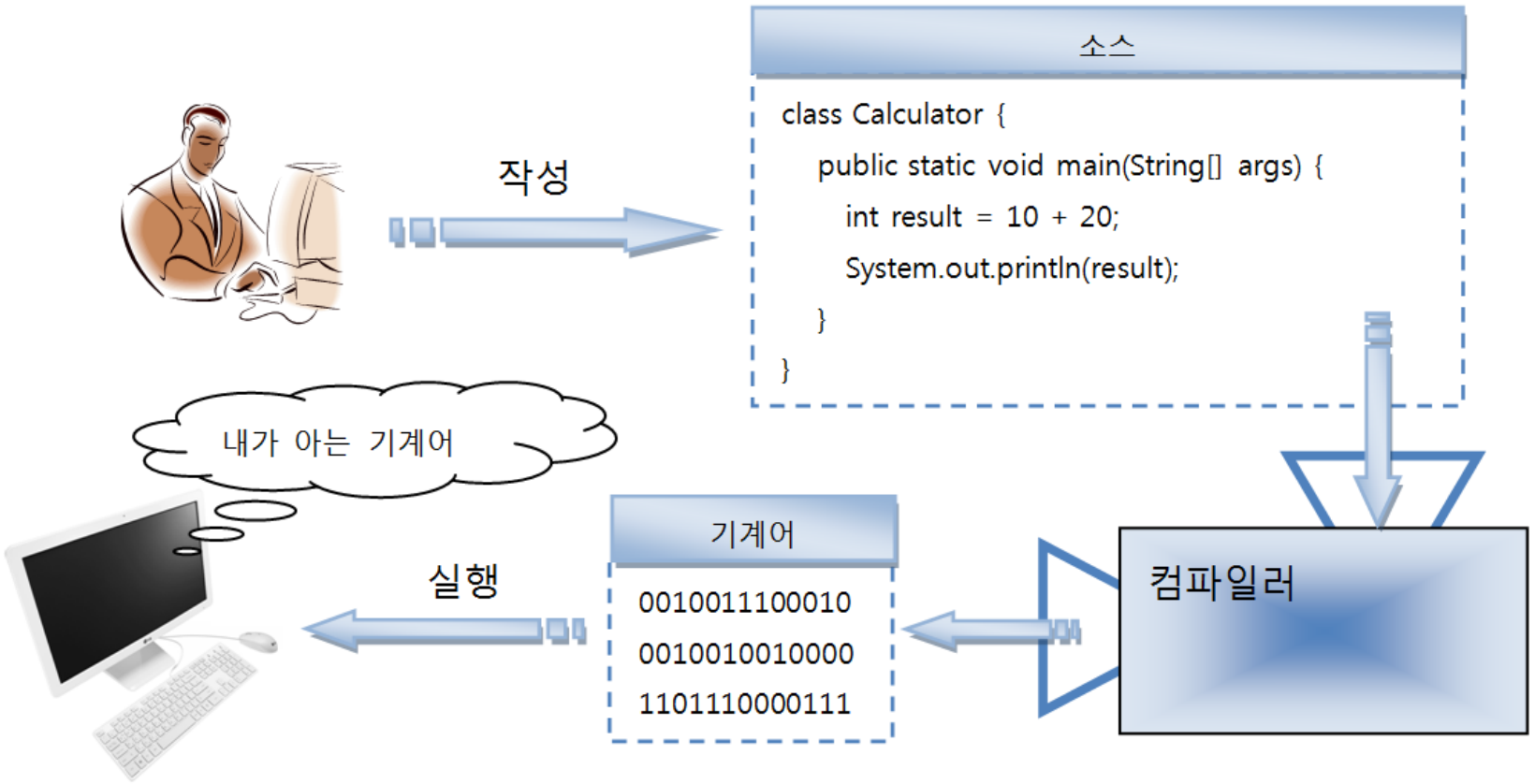
- 1절. 프로그래밍 언어란?
- 2절. 자바란?
- 3절. 자바 개발 환경 구축
- 4절. 자바 프로그램 개발 순서
- 5절. 주석과 실행문
- 6절. 이클립스 설치
- 7절. 풀인원 설치와 실행



1절. 프로그래밍 언어란?

❖ 프로그래밍 언어의 역할은?

- 사람과 컴퓨터의 대화 도움
- 사람의 언어와 기계어 사이에서 다리와 같은 역할
- 고급 언어와 저급 언어로 구분 (p.2)



2절. 자바란?

❖ 1.2.1 자바 소개

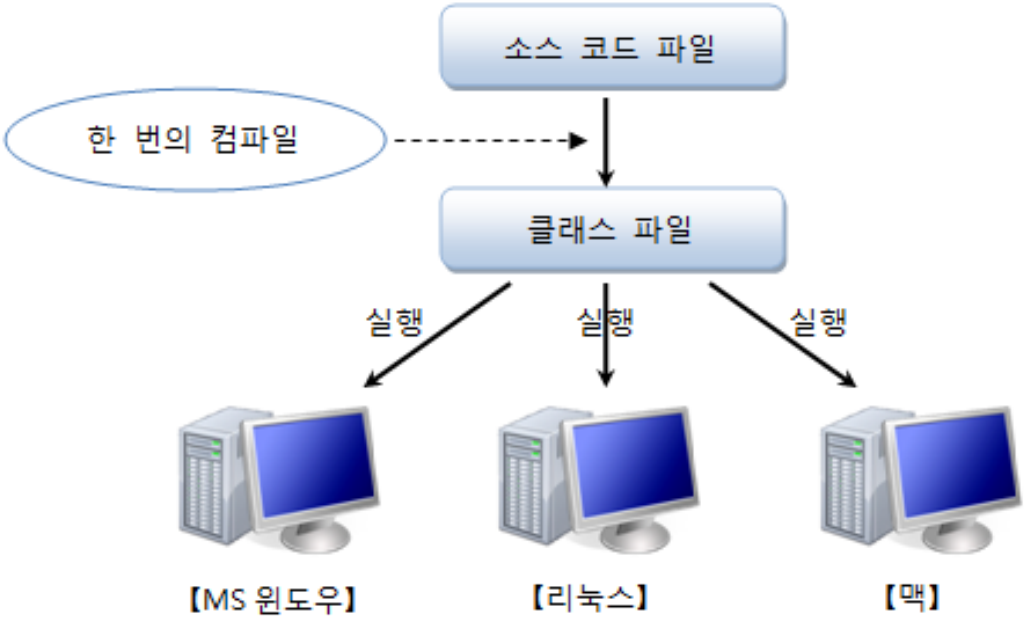
- 1995년 썬마이크로시스템즈(Sun Microsystems)에서 최초 발표한 언어
- 1991년 가전 제품에서 사용할 목적인 오크(Oak) 언어에서부터 시작
- 인터넷 프로그래밍 언어로 발전하면서 자바라는 이름으로 변경
- 2010년 오라클에서 썬을 인수하여 Java 개발,관리,배포 주관



2절. 자바란?

❖ 자바의 특징

- 이식성이 높은 언어

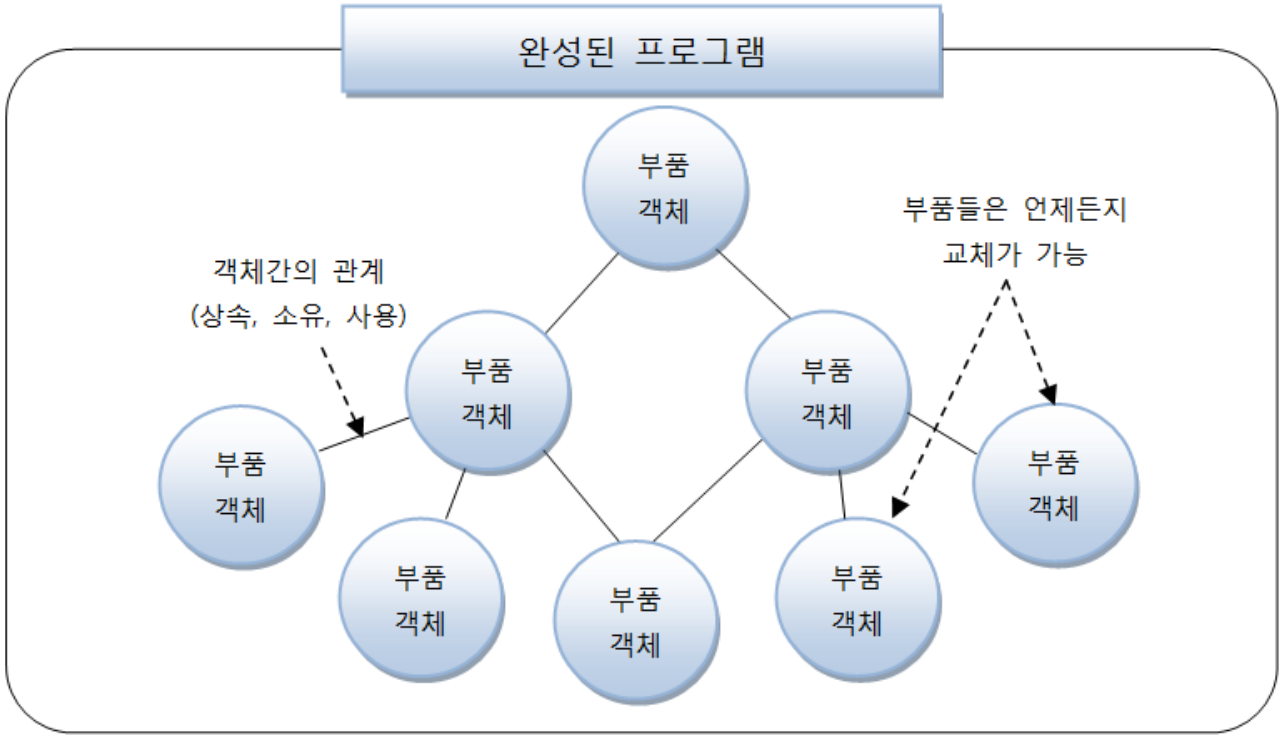


2절. 자바란?

❖ 자바의 특징

- 객체 지향 언어 -> OOP(Object Oriented Programming)란?
  - 부품 객체를 먼저 만들고, 이것들을 조합해 전체 프로그램을 완성하는 기법

- 자바는 처음부터 OOP 개발용 언어로 설계
  - 캡슐화, 상속, 다형성 기능 완벽하게 지원



2절. 자바란?

❖ 자바의 특징

- 함수적 스타일 코딩 지원
  - 함수적 스타일 코딩 방식인 람다식(Lambda Expressions) 지원
    - 코드 간결하게 작성 가능
    - 컬렉션 요소를 필터링, 매핑, 그룹핑, 집계 처리시 주로 사용
- 메모리를 자동으로 관리
  - 사용하지 않는 객체를 자동으로 메모리에서 제거
  - 핵심 기능 코드에 집중할 수 있도록 하는 기능
- 다양한 애플리케이션 개발 가능



2절. 자바란?

❖ 자바의 특징

- 멀티 스레드(Multi-Thread) 쉽게 구현 가능
  - 동시에 여러 가지 작업을 할 경우
  - 대용량 작업을 빨리 처리할 경우

- 동적 로딩(Dynamic Loading) 지원 힙영역 new 예약어
  - 미리 객체를 만들어 놓지 않고 필요한 시점에 동적 로딩해 객체 생성
  - 유지 보수 시 특정 객체만 쉽게 수정 및 교체해 사용

- 막강한 오픈 소스 라이브러리 풍부



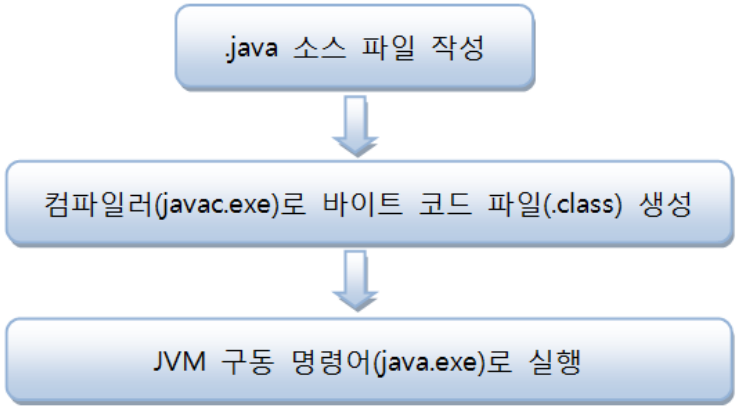
3절. 자바 개발 환경 구축

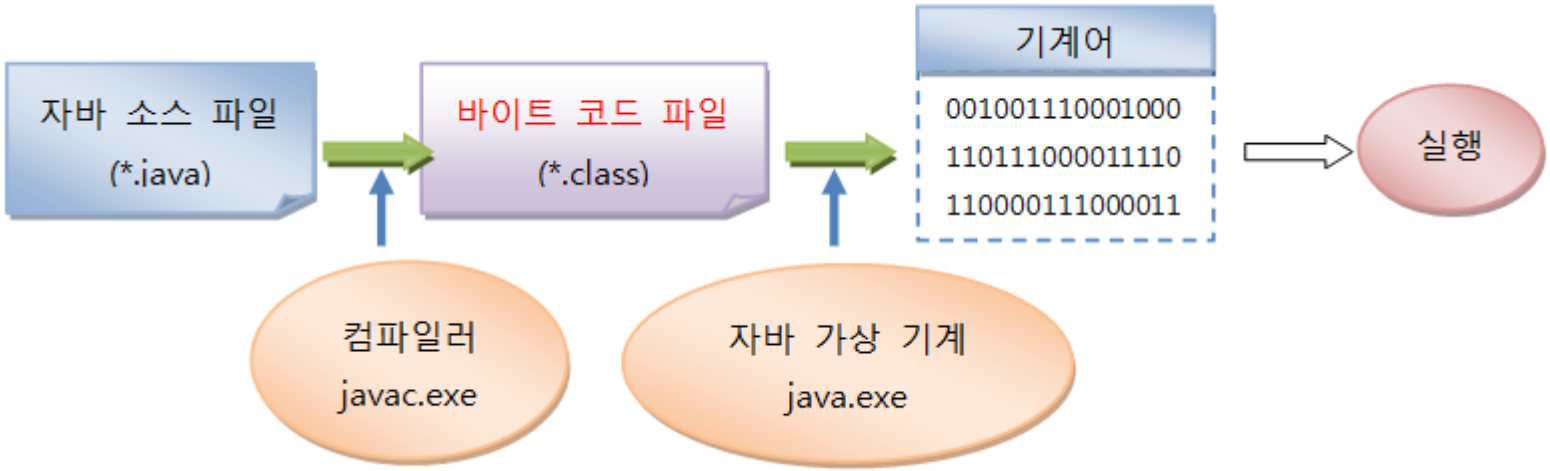
❖ 자바 개발 도구(JDK) 설치

- Java Standard Edition 구현체의 종류
  - JDK(Java Development Kit) = JRE + 개발 도구
    - 자바 프로그램 개발하고 실행하기 위해 반드시 설치
  - JRE(Java Runtime Environment) = JVM + 표준 클래스 라이브러리
    - 자바 프로그램을 실행만 할 경우 설치
- JDK(Java Development Kit) 설치
  - 설치 파일 다운로드 사이트: http://www.oracle.com
- API 도큐먼트
  - JDK에서 제공하는 표준 클래스 라이브러리 설명해 놓은 HTML 페이지들
  - http://docs.oracle.com/javase/버전/docs/api/

4절. 자바 프로그램 개발 순서

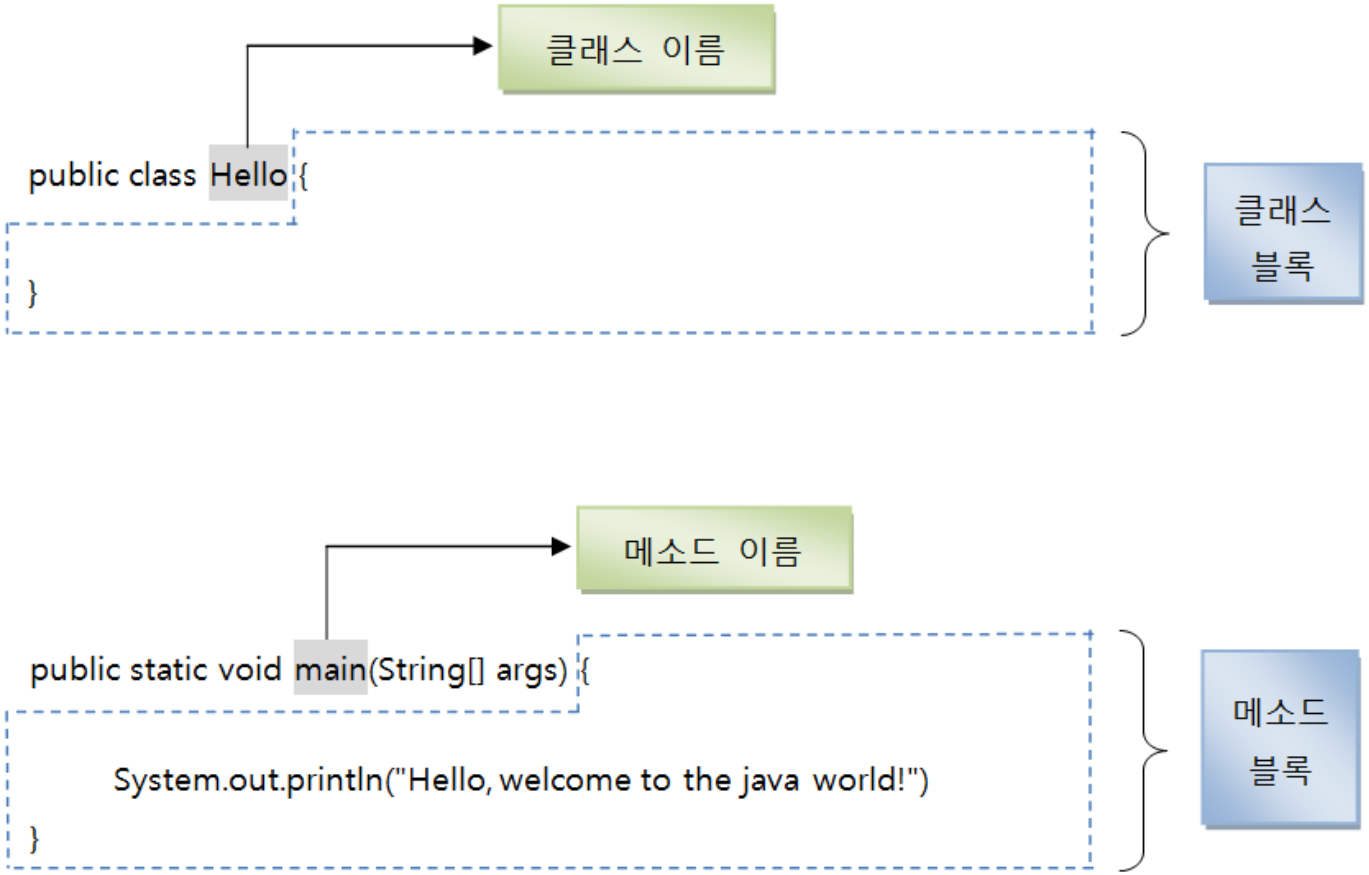
❖ 소스 작성에서부터 실행까지





4절. 자바 프로그램 개발 순서

❖ 프로그램 소스 분석



자바 소스 파일 실습

Hello.java

chap01/src/sec06/exam01/Hello.java

```
2
3  public class Hello {
4      public static void main(String[] args) {
5          System.out.println("Hello, welcome to the java world!");
6      }
7  }
8
```



5절. 주석과 실행문

❖ 주석 사용하기

- 프로그램 실행과는 상관없이 코드에 설명 붙인 것
- 컴파일 과정에서 주석은 무시되고 실행문만 바이트 코드로 번역
- 코드에서 사용하는 주석문의 종류

주석 기호	설명
//	//부터 라인 끝가지 주석으로 처리한다. (행 주석)
/* ~ */	/*와 */ 사이에 있는 모든 범위를 주석으로 처리한다. (범위 주석)



5절. 주석과 실행문

❖ 실행문과 세미콜론(;)

▪ 실행문

- 변수 선언, 값 저장, 메소드 호출에 해당하는 코드
- 실행문 끝에는 반드시 세미콜론(;)을 붙여 실행문의 끝 표시

```
int x = 1;                      //변수 x를 선언하고 1을 저장
int y = 2;                      //변수 y를 선언하고 2를 저장
int result = x + y;             //변수 result를 선언하고 변수 x와 y를 더한 값을 저장
System.out.println(result);     //콘솔에 출력하는 메소드 호출
```

```
int x = 1; int y = 2;
int result =
                x + y;
```



2장. 변수와 타입

이것이 자바다



Contents

1절. 변수

- 변수란?
- 변수의 선언
- 변수의 사용
- 변수의 사용 범위

2절. 데이터 타입

- 기본 타입
- 정수 타입
- 실수 타입
- 논리 타입

3절. 타입 변환

- 자동 타입 변환(Promotion)
- 강제 타입 변환(Casting)



1절. 변수

❖ 변수란?

변수란, 하나의 값을 저장할 수 있는 메모리 공간

❖ 변수의 선언 (p.30)

타입 변수이름

```
int age ;
```

```
double value ;
```

스택 영역 기본타입 변수

1절. 변수

❖ 변수 이름을 위한 명명 규칙(naming convention)

작성 규칙	예
첫번째 글자는 문자이거나 '$', '_' 여야 하고 숫자로 시작할 수 없다. (필수)	가능: price, $price, _companyName 안됨: 1v, @speed, $#value
영어 대소문자가 구분된다. (필수)	firstname 과 firstName 은 다른 변수
첫문자는 영어 소문자로 시작하되, 다른 단어가 붙을 경우 첫자를 대문자로 한다. (관례)	maxSpeed, firstName, carBodyColor
문자 수(길이)의 제한은 없다.	
자바 예약어는 사용할 수 없다. (필수)	책 참조



1절. 변수

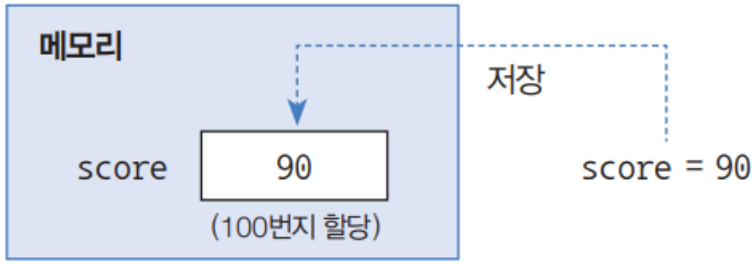
❖ 변수의 사용

▪ 변수값 저장 (p.32~)

```
int score;      //변수 선언
score = 90;     //값저장
```

초기값은 변수를 선언함과 동시에 줄 수도 있다.

```
int score = 90;
```



1절. 변수

❖ 리터럴(literal)

- 소스 코드 내에서 직접 입력된 변수의 초기값
- 소스 코드 내에서 익숙해지는 것이 point !
- 종류 : 정수 리터럴, 실수 리터럴, 문자 리터럴, 문자열 리터럴, 논리 리터럴 (p.32~34)

```
int score = 100;
double  score = 0.2;
char ch = 'A';
String str = "abc";

true  false  (boolean형)
```

부동소수점에 대해서 연구해보기



1절. 변수

❖ 변수의 사용

▪ 변수값 읽기

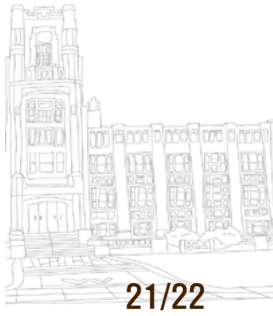
- 변수는 초기화가 되어야 읽기 가능

- 잘못된 코딩의 예

```
int value;                  //변수 value 선언 (초기화 안됨)
int result = value + 10;    //변수 value 값을 읽고 10 을 더한 결과값을 변수 result 에 저장
```

- 맞게 고친 후의 코드

```
int value = 30;             //변수 value 가 30 으로 초기화 됨
int result = value + 10;    //변수 value 값을 읽고 10 을 더한 결과값(40)을 변수 result 에 저장
```



변수값 읽기 – 변수 선언과 초기화

```
1  package sec01.exam01_variable;
2
3  public class VariableExample {
4      public static void main(String[] args) {
5          //10을 변수 value의 초기값으로 저장
6          int value = 10;
7
8          //변수 value 값을 읽고 10을 더하는 산술 연산을 수행
9          //연산의 결과값을 변수 result의 초기값으로 저장
10         int result = value + 10;
11
12         //변수 result 값을 읽고 콘솔에 출력
13         System.out.println(result);
14     }
15 }
```

Console

<terminated> VariableExample [Java Application] C:₩JavaProgramming₩jdk1.8.0_65₩bin₩javaw.exe (2018. 4. 25. 오전 12:26:50)

```
20
```



변수값 읽기 – 변수 선언과 초기화

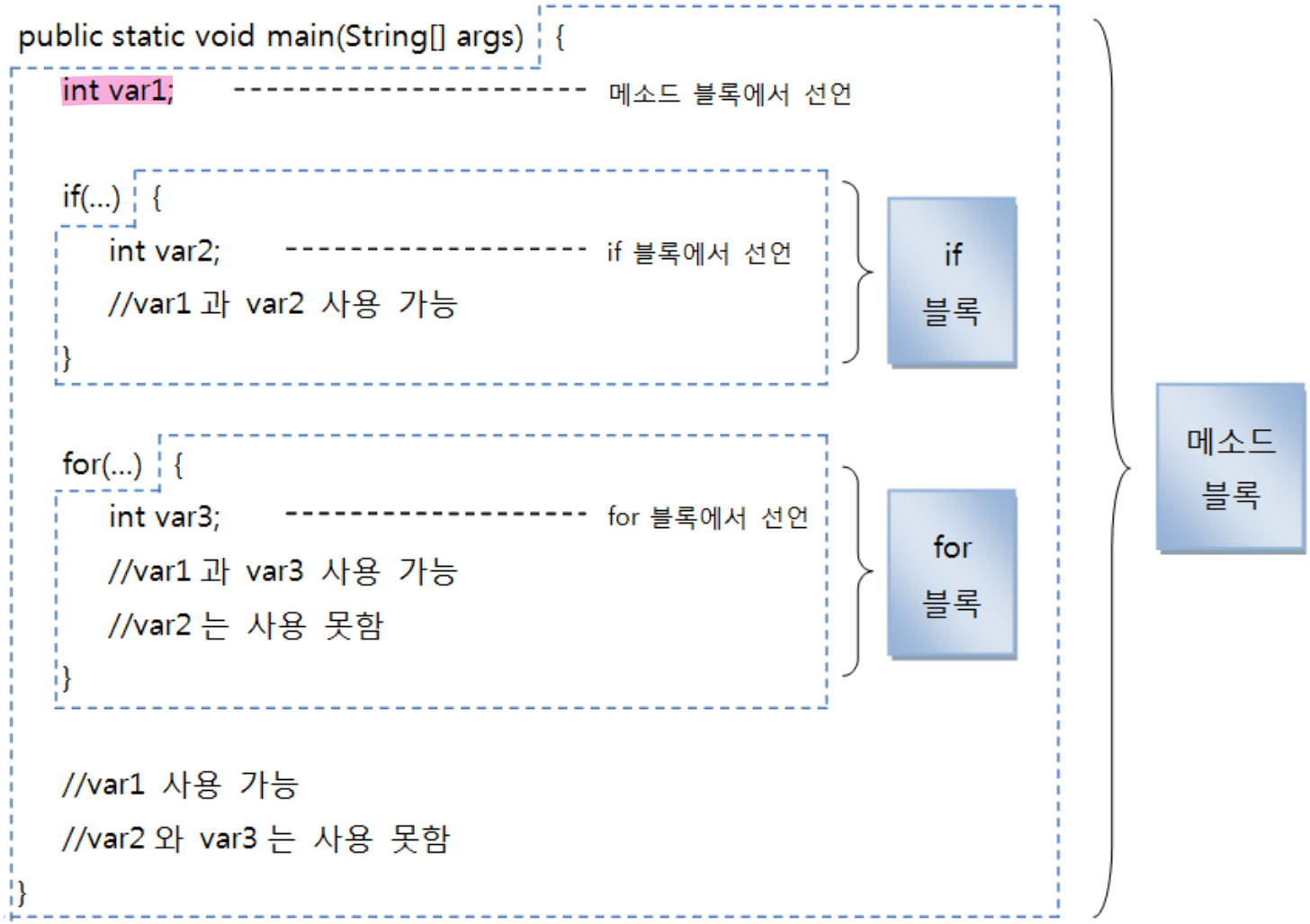
```
1  package ch02.sec01;
2
3  public class VariableUseExample {
4      public static void main(String[] args) {
5          int hour = 3;
6          int minute = 5;
7          System.out.println(hour + "시간 " + minute + "분");
8
9          int totalMinute = (hour*60) + minute;
10         System.out.println("총 " + totalMinute + "분");
11     }
12 }
```

숫자 + 숫자 = 숫자
문자 + 숫자 = 문자

1절. 변수

❖ 변수의 사용 범위

- 변수는 중괄호 블록 {} 내에서 선언되고 사용 (p.35~)



변수의 사용범위(37)

```
1  package sec01.exam01_variable;
2
3  public class VariableScopeExample {
4      public static void main(String[] args) {
5          int v1 = 15;
6          if(v1>10) {
7              int v2;
8              v2 = v1 - 10;
9          }
10         int v3 = v1 + v2 + 5;
11     }
12 }
13
```



2절. 데이터 타입

❖ 기본(primitive) 타입 (p.38~48)

- 정수, 실수, 문자, 논리 리터럴을 직접 저장하는 타입
- 메모리의 최소 기억단위인 bit가 모여 byte 형성

1 byte = 8 bit

0	0	0	0	0	0	0	0

0은 양수쪽에만 있음

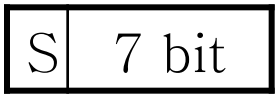
그러면 양수만 다룰거면 메모리 절약차원에서 char로 2의 16승 이하 숫자를 다뤄도 되나 아니면 그냥 short을 쓰나

값의 종류	기본 타입	메모리 사용 크기		저장되는 값의 범위
정수	byte	1 byte	8 bit	2^7~2^7-1 (-128~127)
	char	2 byte	16 bit	0~2^{16}-1 (유니코드: ₩u0000~₩uFFFF, 0~65535)
	short	2 byte	16 bit	-2^{15}~2^{15}-1 (-32,768~32,767)
	int	4 byte	32 bit	-2^{31}~2^{31}-1 (-2,147,483,648~2,147,483,647) 한 21억 5천
	long	8 byte	64 bit	-2^{63}~2^{63}-1
실수	float	4 byte	32 bit	(+/-)1.4E-45 ~ (+/-)3.4E38 7자리
	double	8 byte	64 bit	(+/-)4.9E-324 ~ (+/-)1.7E308 15자리까지 안전하게 사용할 때
논리	boolean	1 byte	8 bit	true, false

정수 타입(40)

종류 \ 크기	1	2	4	8
정수형	byte	short	int	long

byte 타입



$-2^7 \sim 2^7-1$

```
1  package sec02.exam01_byte;
2  public class ByteExample {
3      public static void main(String[] args) {
4          byte var1 = -128;
5          byte var2 = -30;
6          byte var3 = 0;
7          byte var4 = 30;
8          byte var5 = 127;
9          //byte var6 = 128; //컴파일 에러
10
11         System.out.println(var1);
12         System.out.println(var2);
13         System.out.println(var3);
14         System.out.println(var4);
15         System.out.println(var5);
```

Console

<terminated> ByteExample [Java Application] C:₩JavaProgramming₩jdk1.8.0_65₩bin₩javaw.exe (2018. 4. 25. 오전 12:3

```
-128
-30
0
30
127
```

정수 타입(45)

종류 \ 크기	1	2	4	8
정수형	byte	short	int	long

int 타입

S	31 bit

$-2^{31} \sim 2^{31}-1$

```
1  package sec02.exam03_int;
2  public class IntExample {
3      public static void main(String[] args) {
4          int var1 = 10;      //10진수로 저장
5          int var2 = 012;     //8진수로 저장
6          int var3 = 0xA;     //16진수로 저장
7
8          System.out.println(var1);
9          System.out.println(var2);
10         System.out.println(var3);
11     }
12 }
```

Console

<terminated> IntExample [Java Application] C:₩JavaProgramming₩jdk1.8.0_65₩bin₩javaw.exe (2018. 4. 25. 오전 12:5

```
10
10
10
```

정수 타입(46)

종류 \ 크기	1	2	4	8
정수형	byte	short	int	long

long타입 | S | 63 bit | $-2^{63} \sim 2^{63}-1$

```
1  package sec02.exam04_long;
2  public class LongExample {
3      public static void main(String[] args) {
4          long var1 = 10;
5          long var2 = 20L;
6          //long var3 = 1000000000000;
7          long var4 = 1000000000000L;
8
9          System.out.println(var1);
10         System.out.println(var2);
11         System.out.println(var4);
12     }
```

int가 기본형

//컴파일 에러

초기화 전 리터럴 단계에서부터 막힘

Console

<terminated> LongExample [Java Application] C:₩JavaProgramming₩jdk1.8.0_65₩bin₩javaw.exe (2018. 4. 25. 오전 12:52:44)

```
10
20
1000000000000
```

정수 타입(43)

종류 \ 크기	1	2	4	8
정수형	byte	short	int	long

char 타입 16 bit char 0 ~ $2^{16}-1$

```
1  package sec02.exam02_char;
2
3  public class CharExample {
4      public static void main(String[] args) {
5          char c1 = 'A';                  //문자를 직접 저장
6          char c2 = 65;                   //십진수로 저장
7          char c3 = '\u0041';             //16진수로 저장
8
9          char c4 = '가';                 //문자를 직접 저장
10         char c5 = 44032;                //십진수로 저장
11         char c6 = '\uac00';             //16진수로 저장
12
13         int uniCode = c1;               //유니코드 얻기
14
15         System.out.println(c1);
16         System.out.println(c2);
17         System.out.println(c3);
18         System.out.println(c4);
19         System.out.println(c5);
20         System.out.println(c6);
21         System.out.println(uniCode);
22     }
23 }
```

```
A
A
A
가
가
가
65
```

char ch = 'A';

char ch = 'AB'; // 에러

char ch = ''; // 에러

char ch = ' '; // 에러(유니코드32)

char ch = '닭';

3바이트인데 압축 2바이트?



실수 타입(47)

종류 \ 크기	1	2	4	8
실수형			float	double

float 타입

S	E(8)	M(23)

1+ 8+ 23=32 bit = 4 byte

```
1  package sec02.exam05_float_double;
2  public class FloatDoubleExample {
3      public static void main(String[] args) {
4          //실수값 저장
5          double var1 = 3.14;
6          //float var2 = 3.14;
7          float var3 = 3.14F;
8
9          //정밀도 테스트
10         double var4 = 0.12345678901234567890;
11         float var5 = 0.12345678901234567890F;
12
13         System.out.println("var1: " + var1);
14         System.out.println("var2: " + var3);
15         System.out.println("var4: " + var4);
16         System.out.println("var5: " + var5);
17
18         //e 사용하기
19         int var6 = 3000000;
20         double var7 = 3e6;
21         float var8 = 3e6F;
22         double var9 = 2e-3;
23         System.out.println("var6: " + var6);
24         System.out.println("var7: " + var7);
25         System.out.println("var8: " + var8);
26         System.out.println("var9: " + var9);
27     }
28 }
```

기본형이 double

3.140000000000 -> 값손실 정확히 3.14도 아님

//컴파일 에러(Type mismatch)

15~17 반올림 고려해서 15자리까지는 온전히 나오게 하려고

Console

<terminated> FloatDoubleExample [Java Application] C:₩JavaPr

```
var1: 3.14
var2: 3.14
var4: 0.12345678901234568
var5: 0.12345679
var6: 3000000
var7: 3000000.0
var8: 3000000.0
var9: 0.002
```

1+ 11+ 52=64 bit = 8 byte

S	E(11)	M(52)

double 타입

String 타입(47)

```
1  package ch02.sec06;
2
3  public class StringExample {
4      public static void main(String[] args) {
5          String name = "홍길동";
6          String job = "프로그래머";
7          System.out.println(name);
8          System.out.println(job);
9
10         String str = "나는 \"자바\"를 배웁니다.";
11         System.out.println(str);
12
13         str = "번호\t이름\t직업 ";
14         System.out.println(str);
15
16         System.out.print("나는\n");
17         System.out.print("자바를\n");
18         System.out.print("배웁니다.");
19     }
20 }
```

" 문자열로 그대로 넣기

println은 계행이 됨

그냥 print는 ₩n넣어주기

논리 타입(48)

종류 \ 크기	1	2	4	8
실수형			float	double

boolean 타입  true/false

```
1  package sec02.exam06_boolean;
2  public class BooleanExample {
3      public static void main(String[] args) {
4          boolean stop = true;
5          if(stop) {
6              System.out.println("중지합니다.");
7          } else {
8              System.out.println("시작합니다.");
9          }
10     }
11 }
```

Console

<terminated> BooleanExample [Java Application] C:₩JavaProgramming₩jdk1.8.0_65₩bin₩javaw.exe (2018. 4. 25. 오전 1:0

중지합니다.

3절. 타입 변환

❖ 타입 변환

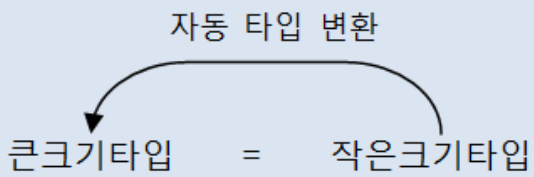
- 데이터 타입을 다른 타입으로 변환하는 것
  - byte ↔ int, int ↔ double
- 종류
  - 자동(묵시적) 타입 변환: Promotion
  - 강제(명시적) 타입 변환: Casting



3절. 타입 변환

❖ 자동 타입 변환 (p.49~51)

- 프로그램 실행 도중 작은 타입은 큰 타입으로 자동 타입 변환 가능



byte(1) < short(2) < int(4) < long(8) < float(4) < double(8)

수를 담을 수 있는 자리수 차이?

Promotion 타입 변환(51)

byte(1) < short(2) < int(4) < long(8) < float(4) < double(8)

```java
package ch02.sec07;

public class PromotionExample {
    public static void main(String[] args) {
        //자동 타입 변환
        byte byteValue = 10;
        int intValue = byteValue;
        System.out.println("intValue: " + intValue);

        char charValue = '가';
        intValue = charValue;
        System.out.println("가의 유니코드: " + intValue);

        intValue = 50;
        long longValue = intValue;
        System.out.println("longValue: " + longValue);

        longValue = 100;
        float floatValue = longValue;
        System.out.println("floatValue: " + floatValue);

        floatValue = 100.5F;
        double doubleValue = floatValue;
        System.out.println("doubleValue: " + doubleValue);
    }
}
```

```
<terminated> PromotionExample [Java Application
intValue: 10
가의 유니코드: 44032
longValue: 50
floatValue: 100.0
doubleValue: 100.5
```

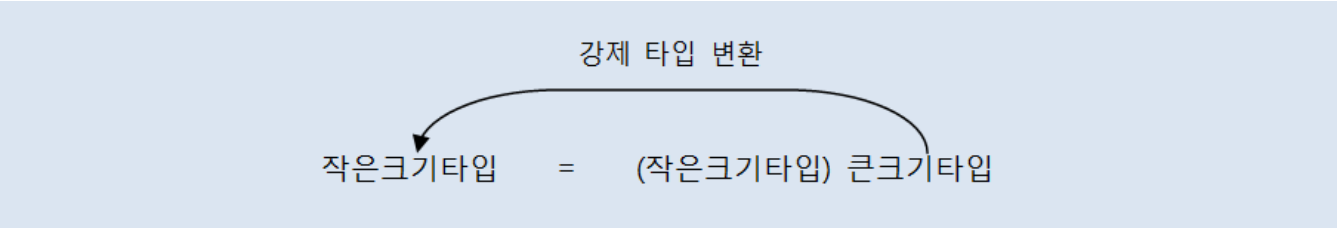
double과 float의 차이는 정밀도
float는 사실상 무한한 값이 가능

float와 double은 정밀도차이

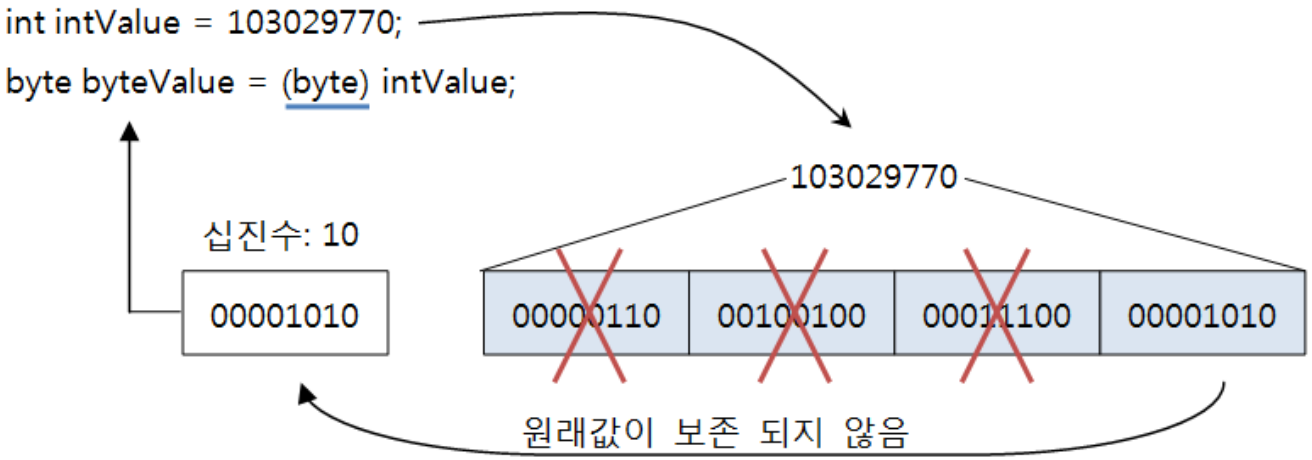
3절. 타입 변환

❖ 강제 타입 변환 (p.52~56)

- 큰 타입을 작은 타입 단위로 쪼개기
- 끝의 한 부분만 작은 타입으로 강제적 변환



- **Ex) int 를 byte에 담기**



Casting 타입 변환(52)

double(8) -> float(4) -> long(8) -> int(4) -> short(2) -> byte(1)

```
1  package ch02.sec08;
2
3  public class CastingExample {
4      public static void main(String[] args) {
5          int var1 = 10;
6          byte var2 = (byte) var1;
7          System.out.println(var2);     //강제 타입 변환 후에 10이 그대로 유지
8
9          long var3 = 300;
10         int var4 = (int) var3;
11         System.out.println(var4);     //강제 타입 변환 후에 300이 그대로 유지
12
13         int var5 = 65;
14         char var6 = (char) var5;
15         System.out.println(var6);    //'A'가 출력
16
17         double var7 = 3.14;
18         int var8 = (int) var7;
19         System.out.println(var8);    //3이 출력
20     }
21 }
```

```
<terminated>
10
300
A
3
```

3절. 타입 변환

❖ 연산식에서 자동 타입 변환 (p.57~59)

- 연산은 같은 타입의 피연산자(operand)간에만 수행
  - 서로 다른 타입의 피연산자는 같은 타입으로 변환
  - 두 피연산자 중 크기가 큰 타입으로 자동 변환

```
int intValue = 10;
double doubleValue = 5.5;

                double 타입으로 자동 변환
double result = intValue + doubleValue;       //result 에 15.5 가 저장
```

  - Ex) int type으로 계산 결과를 얻고 싶다면?
    - Double type 변수를 먼저 int로 변환 후 계산

3절. 타입 변환

❖ 연산식에서 자동 타입 변환 (p.57~59)

```
1  package ch02.sec09;
2
3  public class OperationPromotionExample {
4      public static void main(String[] args) {
5          byte result1 = 10 + 20; //컴파일 단계에서 연산
6          System.out.println("result1: " + result1);
7          byte result2 이런식으로 하면 에러남
8          byte v1 = 10;          byte + short
9          byte v2 = 20;          byte - char 이런경우
10         int result2 = v1 + v2; //int 타입으로 변환 후 연
11         System.out.println("result2: " + result2);
12
13         byte v3 = 10;
14         int v4 = 100;                  이건 더 큰 범위로
15         long v5 = 1000L;
16         long result3 = v3 + v4 + v5; //long 타입으로 변환 후 연산
17         System.out.println("result3: " + result3);
18
19         char v6 = 'A';
20         char v7 = 1;
21         int result4 = v6 + v7; //int 타입으로 변환 후 연산
22         System.out.println("result4: " + result4);
23         System.out.println("result4: " + (char)result4);
24
25         int v8 = 10;
26         int result5 = v8 / 4; //정수 연산의 결과는 정수
27         System.out.println("result5: " + result5);
28
29         int v9 = 10;
30         double result6 = v9 / 4.0; //double 타입으로 변환 후 연산
31         System.out.println("result6: " + result6);
32                                    분자든 분모든 전체든 하나 이상만 쓰면됨
33         int v10 = 1;
34         int v11 = 2;
35         double result7 = (double) v10 / v11; //double 타입으로 변환 후 연산
36         System.out.println("result7: " + result7);
37     }
38 }
```

```
Problems  @ Javadoc  Declara
<terminated> OperationPromotionExam
result1: 30
result2: 30
result3: 1110
result4: 66
result4: B
result5: 2
result6: 2.5
result7: 0.5
```

3절. 타입 변환

❖ 자동 타입 변환 (p.57~59)

```java
package ch02.sec09;

public class StringConcatExample {
    public static void main(String[] args) {
        //숫자 연산
        int result1 = 10 + 2 + 8;
        System.out.println("result1: " + result1);

        //결합 연산
        String result2 = 10 + 2 + "8";
        System.out.println("result2: " + result2);

        String result3 = 10 + "2" + 8;
        System.out.println("result3: " + result3);

        String result4 = "10" + 2 + 8;
        System.out.println("result4: " + result4);

        String result5 = "10" + (2 + 8);
        System.out.println("result5: " + result5);
    }
}
```



3절. 타입 변환

❖ 자동 타입 변환 (p.57~59)

```
1  package ch02.sec10;
2
3  public class PrimitiveAndStringConversionExample {
4      public static void main(String[] args) {
5          int value1 = Integer.parseInt("10");
6          double value2 = Double.parseDouble("3.14");
7          boolean value3 = Boolean.parseBoolean("true");
8
9          System.out.println("value1: " + value1);
10         System.out.println("value2: " + value2);
11         System.out.println("value3: " + value3);
12
13         String str1 = String.valueOf(10);
14         String str2 = String.valueOf(3.14);
15         String str3 = String.valueOf(true);
16
17         System.out.println("str1: " + str1);
18         System.out.println("str2: " + str2);
19         System.out.println("str3: " + str3);
20     }
21 }
```



3절. ScannerExample.java

```java
package ch02.sec13;

import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);

        System.out.print("x 값 입력: ");
        String strX = scanner.nextLine();
        int x = Integer.parseInt(strX);

        System.out.print("y 값 입력: ");
        String strY = scanner.nextLine();
        int y = Integer.parseInt(strY);

        int result = x + y;
        System.out.println("x + y: " + result);
        System.out.println();

        while(true) {
            System.out.print("입력 문자열: ");
            String data = scanner.nextLine();
            if(data.equals("q")) {
                break;
            }
            System.out.println("출력 문자열: " + data);
            System.out.println();
        }

        System.out.println("종료");
    }
}
```

순수문자를 숫자로 바꾸려고 할 때 NaN

값을 키보드로 받기

enter칠 때까지

vs nextInt() 보다는 string형태로 받고 작업 한 후에 형변환 하는 것이 유리하다.

java.lang.Object
java.util.Scanner

String next()

계행

